Theoretische Elektrotechnik
Universität Paderborn

python/NumPy fundamentals, modeling on real physical problems

return: 27.10.2026 9:00am (20 points)

Create a Python script file (.py) for the following problems:

1. **Use "numpy.diag" function to generate the matrix (2P):**
$$\begin{pmatrix} 2 & -3 & -1 & -5 \\ 3 & 2 & -3 & -1 \\ 1 & 3 & 2 & -3 \\ 5 & 1 & 3 & 2 \end{pmatrix}$$

2. **Comparison operators (1P):**
   (a) Define a 1×5 vector a with elements equal to $(5, 0, -2, 1, 8)$ and a 1×5 vector b with elements equal to $(3, 4, 0, 1,$ *last number of your student id*$)$. (0.5P)
   (b) Compare the corresponding values of a and b element-wise using "Less than", "Greater than" and "Equal to". Return the result to vector c. (0.5P)

3. **Flow control (4P):**
   (a) Define a 1×4 vector a for *the last four numbers of your student ID.* (1P)
   (b) Use a "for" loop to find the minimum number in a. (1P)
   (c) Use "while" statement to find the maximum number in a. (1P)
   (d) Use "for" statement to compute $y = sin(x)$ from $x = 0$ to π at increment of $\pi/20$ and plot the curve. (1P)

4. **Functions and data input/output (8P):**
   (a) In the main script file, write an inline-function to calculate $2x^2 - y^4 + 3z^3$. Call your inline-function in the main script file, and write out the solution of this expression for the values of x, y, and z equal to the first three digits of *your matriculation number*, respectively. (1P)
   (b) Write a function (.py) that solves a system of linear equations $A * x = b$ using the Gaussian elimination algorithm. The function takes two input parameters - a square matrix A, and vector b, and the output is the solution vector - x. In the main script file, call your function and use your matrix A and vector b generated in PS1, Ex.5 to check your solution; write out the solution in the main script file. (7P)

5. **Plotting Tools (1P):**
Consider the function $x = cos(t)$, $y = sin(t)$, $z = sin(t)$. Use "matplotlib 3D plotting" to plot the curve in 3D space, add mesh grid and axis label "x", "y", "z" on the figure. (1P)

6. **Modeling on real physical problems (4P):**
For all classifications mentioned in the lecture ((1) physical, (2) abstract (paper, computer etc), (3) static, (4) dynamic, (5) real-time, (6) interactive, (7) (time-) continuous, (8) discrete (iterative or event driven), (9) realistic, (10) simplistic, (11) natural, (12) artificial, (13) imagined system, (14) simple, (15) complex, (16) deterministic, (17) stochastic, (18) known input, (19) statistical properties of input known, (20) unknown input), think of one model which falls into that category. Models may reappear, but should be clear and specific.
Example: "pendulum from lecture: 2,4,7,10 (damping neglected), 11, 14, 16, 18".

Hints and Notes:
In this exercise, you are required to use:

- NumPy for vectors and matrices
- matplotlib for plotting

Add the following lines at the top of your code:

- import numpy as np
- import matplotlib.pyplot as plt

Instructions for the submission:
For homework submission, please submit the following **three files**:

1. One main Python script (.py) containing Exercises 1–6, named:

   `ps_2_ex_1to6_your_matriculation_number.py`

2. One separate Python file (.py) containing the function for Exercise 4(b), named:

   `ps_2_ex_4b_your_matriculation_number.py`

3. One Jupyter Notebook (.ipynb) containing Exercises 1–6, including all code, results, and plots, named:

   `ps_2_your_matriculation_number.ipynb`

Important:
Make sure to run all notebook cells before submission. All results and plots must be visible in the notebook.